

延迟工程与 Tick-to-Trade 系统设计

一场推演:那些被挑战、被反驳、再拧准的地方,才是精华

Robert (Bo) Hou · Robert's Technical Notes · 2026 年 6 月

摘要与读法。这份报告刻意保留推演的来回过程,而非只给结论——因为真正的理解藏在“一个直觉被挑战、然后拧准”的那一步里。全篇采用三段式:直觉 » 一个初始判断(往往对一半)、张力 » 它在哪里站不住/被挑战、收敛 » 拧准后的结论。第一部分(延迟工程)是消灭尾部尖峰的武器库;第二部分(系统设计)把武器连同前三座山(无锁、cache、零开销)装成完整的 tick-to-trade 链路。贯穿全篇一句话:HFT 优化“确定性”(最坏延迟有上界),不优化“平均速度”;能连续、能预测、能复用,就别动态分配、别指针追逐、别把可提前确定的代价留到运行时——而系统的最终验收,不靠公式,靠回放最凶的历史行情。

第一部分 · 延迟工程:消灭尾部尖峰

1.1 世界观:确定性,而非平均

为什么 HFT 关心 p99.9 而非平均? 价值在“最坏情况下能否抢到/不亏”。一次尖峰恰好爆发在大行情(信息量最大、竞争最激烈、机会最大)的时刻——那一刻系统负载也最高、最易卡顿;而代价不对称:平均快带来线性小收益,一次尖峰错失/逆向成交带来非线性大损失。故优化最坏延迟的上界与 p99.9/p99.99。确定性 > 平均速度,这是延迟系统与吞吐系统最根本的分野。测量上的经典坑是 Coordinated Omission:若测量是“发一个→等回来→记录→再发”,系统卡顿时连发请求都被卡住,漏记了大量本该很慢的样本,尾部被严重低估;解法是按固定速率发送。工具:HdrHistogram + rdtsc。

1.2 PAUSE 指令:忙等为何要“礼貌地空转”

直觉 » 忙等循环疯狂空转,PAUSE 大概是告诉 CPU“别做无用的预取(prefetch)”。

张力 » 机制不是 prefetch。空转循环 while(flag==0){} 会让 CPU 投机执行一大堆对 flag 的读(load)塞满流水线;当 flag 真被别的核改动,这些投机 load 全成旧值,必须 flush 重来——这叫 memory-order violation 惩罚。不是“预取白做”,是“投机 load 白做要重罚”。

收敛 » PAUSE 告诉 CPU“我在自旋等待”,于是 (a) 减少投机 load 堆积、省掉 flag 变化时的 memory-order-violation flush,(b) 把物理核的执行资源让给超线程兄弟。核心词是投机 load + memory-order violation,不是 prefetch。

直觉 » “一个核跑两个线程”不是主流吧,而且线程本来就能跨核执行。

张力 » 超线程(SMT)真实且主流——12 物理核 / 16 逻辑处理器的不对称,正是部分核开了超线程(一物理核 = 2 逻辑核,共享执行单元)。而“线程跨核”是 OS 默认行为,但对 HFT 是灾难:迁到新核后 L1/TLB 冷,拖尾 miss。

收敛 » 所以 PAUSE“让超线程兄弟”有实义(两线程共享一个物理核的 ALU)。而 HFT 用 CPU pinning + 核隔离(isolcpus)禁止迁移、独占一核;为确定性,甚至关超线程 / 空出兄弟核——宁可浪费半核,换确定性。

1.3 尖峰敌人清单 —— 每个对应一个武器

① 动态内存 new/malloc:遍历 free list(双向链表)+ 记账 + 锁 + 偶发 syscall,且分配物散落堆各处后续 cache miss → 预分配 arena/pool,热路径不 new。② 缺页 page fault:数据不在物理内存、读硬盘、~百万周期 → mlockall 锁内存 + 预热(pre-fault)。③ 上下文切换:切回来 TLB/cache 冷、拖尾 miss → pinning + 核隔离。④ NUMA 跨节点:走 CPU 互联、延迟高 1.5–2 倍 → 线程与数据绑同一节点。每个解法都是前几座山机制的应用。

1.4 Kernel Bypass 与“轮询一定更快吗”

内核网络栈四处慢:中断(每包中断→上下文切换风暴)、syscall + 内核/用户态切换、数据拷贝(内核缓冲→用户缓冲、CPU 搬 + 污染 cache)、通用协议栈(校验/重组/路由)。逐个绕过:中断→poll mode、拷贝→zero-copy、协议栈→用户态精简栈。代表 DPDK / Solarflare onload / io_uring。

直觉 » 既然轮询(poll)取代中断,那轮询当然一定比中断延迟低。

张力 » 不一定。轮询快的前提是 CPU 专职轮询(独占核忙等、轮询间隔趋近 0)。若CPU 不专职、轮询夹在别的活中间,两次轮询间隔了处理时间,包可能要等,反而不如“能立刻打断”的中断。

收敛 » HFT 恰好满足前提(舍得烧核),故用轮询、又快又稳。通则:每个“更快”都有前提,脱离前提不成立——正因如此,kernel bypass + 忙等 + 核隔离必须成套上,单拆一个不一定有效。

直觉 » 不用中断,就设一个 flag,让网卡写、CPU 每次轮询读这个 flag,网卡和 CPU 就解耦了。

张力 » 这个设计几乎就是 DPDK 的 descriptor ring:实际是一个环(多个槽)而非单个 flag,网卡在描述符的 status 位标“有新包”,CPU 轮询这些位。而这个 ring 就是无锁 SPSC 环形缓冲区(网卡生产、CPU 消费)。

收敛 » 关键的同步细节正是无锁那座山:“看到 status 就绪时,包数据必已 DMA 完成”= release/acquire 内存序(SPSC 报告里的对偶边)。至此无锁(ring + 内存序)与延迟工程(忙等 + 消灭中断)在网卡处缝合——你独立设计出的正是 poll-mode driver 的核心。

第一部分总纲。 延迟工程每一项——忙等(烧一核)、核隔离(独占一核)、关超线程(浪费半核)、kernel bypass(独占网卡 + 自写栈)、锁内存(占着不还)——本质都是同一笔交易:牺牲资源利用率与通用性,换最坏情况的确定性,与吞吐系统方向恰好相反。

第二部分 · Tick-to-Trade 系统设计

2.1 需求拆解:把“峰值百万/秒”拆成三层

给定单交易所连接、峰值 100 万消息/秒、每条二进制 32-48 字节、策略处理时间外生上界 $10\mu\text{s}$ /条。“百万/秒”必须拆:平均 vs 峰值(按峰值设计,峰值 = 机会最大 + 最易卡顿);突发性(成簇到达,方差远大于泊松,故必须有缓冲);单条预算(峰值 $\Rightarrow$ 平均每条 $1\mu\text{s}$,枪毙一切 syscall/malloc/锁/miss 密集结构,前四座山由此从“选项”变“必需”)。

2.2 容量:吞吐 $\neq$ 延迟,利用率悬崖,平均无用

直觉 » 单核全链路 $10+1+2 = 13\mu\text{s}$ 一条,故一秒一核最多处理 76923 条。

张力 » 混淆了延迟与吞吐。链路是流水线:收包、读取、策略是不同阶段可重叠,处理第 1 条策略时网卡已在收第 2、3 条。吞吐由最慢一级(瓶颈)决定,不是把总延迟当除数。

收敛 » 策略 $10\mu\text{s}$ 是瓶颈 $\Rightarrow$ 单核吞吐 $= 1/10\mu\text{s} = 10$ 万/秒(非 76923)。峰值 100 万/秒 $\Rightarrow$ 稳定性要求核数 $c > \lambda/\mu = 10$ 。

直觉 » 32 核、利用率 31%,排队论算出平均等待 $0.00104\mu\text{s}$,基本是 0,够了。

张力 » 平均是最没用的指标(你自己第一部分就说过看尾部)。而且 M/M/c 假设泊松,真实行情是突发。你的“限制方差”直觉才是正解方向——控制的是尾部,不是平均。

收敛 » 算真实的 p99:16 核平均仅 $0.096\mu\text{s}$,但 p99.9 达 $6.7\mu\text{s}$ (100 倍);32 核泊松下连 p99.99 排队都 ≈ 0 。超配到 31% 利用率买的不是更低平均(16 核平均已 ≈ 0),是突发下的尾部安全边际。

核数	利用率	平均 W_q	p99.9(泊松)
11	91%	$6.82\mu\text{s}$	—(勉强稳定)
16	62%	$0.096\mu\text{s}$	$6.7\mu\text{s} \leftarrow$ 平均骗人
20	50%	$0.0037\mu\text{s}$	$1.3\mu\text{s}$
32	31%	$\approx 0\mu\text{s}$	$\approx 0\mu\text{s}$

利用率悬崖。11 核(91%)到 16 核(62%),排队暴跌约 70 倍——利用率越近 100%,延迟越非线性爆炸。这是排队论最重要的直觉,也是超配核数的理由:远离悬崖,突发来了才不逼近满载。

2.3 泊松 vs 突发:细分时间能建模突发吗?

这是全篇来回最多、最精华的一段——反复逼问“真实 p99.9 到底能不能算”。

直觉 » 行情虽突发,但在 $1\mu\text{s}$ 这么细的粒度看“一次来一个”已够细;若不够就再细到纳秒。微积分思想,细分时间单元就能用泊松逼近突发。

张力 » 泊松是“无记忆(memoryless)”的——事件独立到达,过去不影响未来。而突发的本质是“有记忆/自激”:来了一簇让接下来更容易再来一簇(成群结队)。细分时间只把粒度变细,每个小单元内仍是无记忆泊松,拼起来仍缺“跨段自相关”。记忆性是结构问题,不是分辨率问题。

收敛 » 细分 $\Rightarrow$ 非齐次泊松(NHPP,确定的时变率),仍假设“突发时刻已知 + 单元内独立”;真是随机自激的 Hawkes 过程(方差远大于泊松)。用泊松估突发尾部会系统性低估 p99.9。

直觉 » 那就用傅里叶思想:任意强度曲线 $\lambda(t)$ 都能用各种频率逼近,数学上一定有人这么做。

张力 » 你逼近的是“一条确定的 $\lambda(t)$ 曲线”(= NHPP)。但真实 $\lambda(t)$ 是随机的(Cox)且自激的(Hawkes)——它不是一条能预先画出的曲线,而是由自身历史递归生成、每次实现都不同的随机对象。傅里叶能逼近“已实现的一条”,给不出“对所有随机路径成立的 p99.9”。

收敛 » “事后对着已发生的两秒画 $\lambda(t)$ ”能逼近 (对),但容量规划是事前的,明天的簇在不同时刻/强度出现。逼近昨天 $\neq$ 保证明天。故真实 p99.9 只能回放真实历史行情(其中天然带真实簇结构)实测——排队论只用于“选多少核起步”与“看清悬崖”。

这段的收敛。在你设定的 M/M/c 模型内,微秒级泊松的计算无懈可击,选核数(≥ 11 稳定、32 留余量)也完全够用——这点你是对的,不是巧合。分歧只在“模型 vs 真实”:真实突发是自激的,任何粒度的平滑泊松都低估尾部。两者不矛盾:泊松估算用于选型,回放历史用于验收。

2.4 并行:SPMC 还是分片 SPSC?

直觉 » 多核处理,那就 SPMC——32 核共读一个队列。SPMC 本身有令牌(token)能无锁做,没问题。

张力 » SPMC 确实能用 per-slot token 无锁(你 MPMC 报告里的机制)。但无锁 $\neq$ 无争用:32 个消费者 CAS 同一个 head_ $\Rightarrow$ 那条 cache line 在各核 L1 $\leftrightarrow$ L3 间 ping-pong。无锁只保证“争用中总有人前进”,不保证“不争用”。而且随机分发导致同标的乱序。

收敛 » 你那句“为多消费者去增加写的复杂度,有点奇怪”正是正解方向:别用 SPMC,用分片 SPSC——按 symbol hash 分成 32 个独立队列,每核独占一个。单消费者直接 head_++,无 CAS、无争用;同标的永进同一核 $\Rightarrow$ 天然保序。用架构分片消除争用,优于用无锁队列硬扛争用。

争用点澄清:SPMC 的 buffer 有很多槽(可 $\gg 32$,分散在多条 cache line,不是争用点);争用只在那一个被 32 核抢的 head_ 索引上。buffer 再大也没用,瓶颈是那一个共享 head。分片后每个队列的 head 只有一个核碰,一直热在它的 L1。

2.5 DDIO:队列在内存还是 L3?

这也是来回多次才闭环的一段——核心是“逻辑地址”与“物理位置”两个层面的分离。

直觉 » 既然 NIC 用 DDIO 把数据直接写 L3、不经过内存,那为什么还在内存上设计 32 个队列?为什么不直接在 L3 上分发?L3 共享,32 个独立队列不就是 L3 的 32 个分区?

张力 » 程序不能直接操作 L3。CPU 核只能用内存地址寻址,cache 对程序透明、不可编程;L3 每行由 tag 绑定一个内存地址,你没有“L3 地址”可用。所以数据必须有内存地址,队列必须用内存地址组织——这正是 NIC 与 CPU 通信的共同约定。L3 也不预先分区,是按访问动态缓存。

收敛 » 但你咬住的“最后数据还是从 L3 读”完全正确——这里根本没有分歧。“队列在内存”说的是地址归属(户籍),“数据从 L3 读”说的是物理位置(当前所在),是同一份数据的两个层面。load [addr] 用内存地址读、数据从 L3 命中,同一个动作。DDIO 省的是“先落 DRAM 再被读回”的往返,不是让数据脱离内存地址。

直觉 » 那 32 个独立队列,映射回 L3 缓存上应该也是独立的吧?

张力 » 方向对,但“逻辑独立”不自动等于“缓存独立”。若两个队列的 head/tail 在内存里挨太近、落在同一条 cache line,即使逻辑独立,也会 false sharing ping-pong(第二座山)。

收敛 » 前提是每个热变量用 alignas(64) 对齐到独立 cache line。分片消除了“多核抢同一 head”的争用,还需 alignas 防“不同队列的 head 挤同一 line”的 false sharing——两个都做,才真正缓存独立。

2.6 Buffer 定容与突发击穿降级

直觉 » 32 核平均等待 ≈ 0 ,理论上 buffer 不用很大;一条 cache line 给一个核装东西也够了。

张力 » 两个尺度别混:cache line(64 字节)是 false sharing 对齐单位(head/tail 各占一条);buffer 大小是突发缓冲单位,由“缓冲多少条消息”定。一条 line 只装 16 个 float / 1 条 64 字节消息,扛不住突发。

收敛 » 理论 buffer(按平均) ≈ 0 ,实际要大几十倍防突发击穿——按最坏突发的瞬时堆积定;但不能无脑大(超 L2 变慢、深队列延迟大)。折中:够扛历史最凶突发 + 仍进 L2,靠回放历史看瞬时堆积峰值定。

直觉 » 再好的设计也防不住黑天鹅击穿,那队列满了怎么办?丢老不丢新——这就是 LRU cache 了。

张力 » 丢老不丢新 (对) (行情会过期,旧报价决策 = 逆向选择亏钱,新鲜度即价值)。但不是 LRU:LRU 按“访问历史”淘汰求命中率,这里按“到达时序/新鲜度”淘汰。更准的名字是过期数据淘汰。

收敛 » 最佳形式是 conflation(合并):同标的新报价直接覆盖队列中旧报价,突发时同标的一串更新折叠成最后一条,队列不爆。conflation 按 symbol 折叠,逻辑是 hash;但 symbol id 通常是密集小整数,故直接用数组下标索引(latest[symbol])而非 std::unordered_map——O(1)、无冲突、无 malloc、cache 友好。

2.7 为何“框图相通”却仍是难题

直觉 » 这一路推下来,系统设计感觉没有很大难度,不像数学证明想半天。是不是它就等于“普通大数据系统 + 排队论 + 实时性”?

张力 » 推得顺,是因为思维类型不同 + 你有备而来:数学证明是收敛思维(逼出唯一反直觉构造,故卡);系统设计是发散/权衡思维(在已知选项里判断,故顺)。而且积木(无锁、cache、零开销)你前四座已烧好,这里只是组装。

收敛 » 架构骨架确与大数据相通,但微秒级 +

确定性把每个组件逼成完全不同的实现:同一个“队列”,大数据是 Kafka(毫秒/磁盘/网络),HFT 是无锁 SPSC(纳秒/cache/单核),差 6

个数量级。会画框图的多,能让每个组件在纳秒级确定性工作的少——那正是前四座山。

组件	普通大数据	HFT
延迟目标	毫秒~秒,优化平均/吞吐	微秒~纳秒,优化最坏/确定性
队列	Kafka(磁盘/网络/毫秒)	无锁 SPSC 环形(纳秒)
分片	跨机器(网络)	跨核(cache line)
内存	GC,停顿几十 ms 无妨	C++ 预分配,禁 GC/malloc/缺页
网络	内核 TCP 栈	kernel bypass
容错	副本/重试/最终一致	几乎不重试(重试=延迟=错失)

收官。系统设计“看着不难”,是因为它站在前四座山的肩膀上:积木已烧好、验证过,这里只是组装。五座山最终收束到同一句话:能连续、能预测、能复用,就别动态分配、别指针追逐、别把可提前确定的代价留到运行时;而最终验收,不靠公式,靠回放最凶的历史行情。